

React & React Native

*Guide de référence — des fondamentaux React
aux spécificités React Native*

Préparé pour un profil Symfony / JS-jQuery / React (Preact)

Partie 1 — React : les fondamentaux et bonnes pratiques actuelles

1.1 Le modèle mental de React

React repose sur une idée simple mais qui change la façon de penser l'UI par rapport à jQuery : **l'interface est une fonction de l'état** (`UI = f(state)`). Plutôt que de manipuler le DOM impérativement ("trouve cet élément, change sa classe, ajoute ce texte"), on décrit *ce que l'UI doit être* pour un état donné, et React se charge de calculer les changements DOM nécessaires.

C'est le changement de paradigme le plus important venant de jQuery : on ne dit jamais "fais ceci au DOM", on dit "voici à quoi ressemble l'UI maintenant", et React fait la différence (le *diffing*) avec ce qui était affiché avant.

```
// jQuery : impératif, on dit COMMENT faire
$('#compteur').text(valeur);
$('#bouton').on('click', () => {
  valeur++;
  $('#compteur').text(valeur);
});

// React : déclaratif, on dit CE QUI doit être affiché
function Compteur() {
  const [valeur, setValeur] = useState(0);
  return (
    <div>
      <span>{valeur}</span>
      <button onClick={() => setValeur(valeur + 1)}></button>
    </div>
  );
}
```

Tu as déjà cette intuition via Preact, donc cette partie sera surtout un rappel structuré plutôt qu'une découverte.

1.2 Composants fonctionnels et Hooks (le standard actuel)

Les **class components** (avec `this.state`, `componentDidMount`, etc.) sont aujourd'hui considérés comme legacy. L'écosystème React moderne (et tout React Native) repose sur les **composants fonctionnels + Hooks**. Si ton expérience Preact date d'avant cette transition, c'est le point le plus important à mettre à jour.

useState — état local

```
import { useState } from 'react';

function Formulaire() {
  const [nom, setNom] = useState('');
  const [age, setAge] = useState(0);

  return (
    <div>
      <input value={nom} onChange={(e) => setNom(e.target.value)} />
      <input type="number" value={age} onChange={(e) => setAge(Number(e.target.value))} />
    </div>
  );
}
```

Point important : `setNom('...')` ne modifie pas `nom` immédiatement et de façon synchrone — ça programme un **nouveau rendu** du composant avec la nouvelle valeur. Ne jamais s'attendre à lire la valeur à jour juste après un `setState`, dans la même fonction.

useEffect — effets de bord (équivalent du cycle de vie)

C'est l'équivalent unifié de `componentDidMount`, `componentDidUpdate`, et `componentWillUnmount` des class components.

```
import { useEffect, useState } from 'react';

function ProfilUtilisateur({ userId }) {
  const [utilisateur, setUtilisateur] = useState(null);

  useEffect(() => {
    // S'exécute après le premier rendu, ET chaque fois que userId change
    fetch(`/api/utilisateurs/${userId}`)
      .then(res => res.json())
      .then(data => setUtilisateur(data));
  }, [userId]); // <- le tableau de dépendances

  if (!utilisateur) return <p>Chargement...</p>;
  return <p>{utilisateur.nom}</p>;
}
```

Le tableau de dépendances est le point le plus piégeux de tout React moderne. Règles à retenir : - `[]` (vide) → l'effet ne s'exécute qu'au montage, jamais après (équivalent `componentDidMount` seul) - `[userId]` → l'effet se ré-exécute chaque fois que `userId` change - Pas de tableau du tout → l'effet s'exécute à **chaque** rendu (rarement ce qu'on veut, source classique de boucles infinies ou de performances dégradées)

Piège fréquent : oublier une dépendance utilisée dans l'effet. ESLint (plugin `eslint-plugin-react-hooks`) te signale ça automatiquement — ne jamais ignorer ce warning sans comprendre pourquoi.

Nettoyage (équivalent `componentWillUnmount`) :

```
useEffect(() => {
  const interval = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(interval); // fonction de nettoyage
}, []);
```

useContext — partager un état sans tout faire passer par props

Utile pour éviter le "prop drilling" (passer une prop à travers 5 niveaux de composants juste pour qu'elle arrive au bon endroit) :

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState('clair');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
    </ThemeContext.Provider>
  );
}

function Header() {
  const { theme } = useContext(ThemeContext); // accès direct, peu importe la profondeur
  return <div className={theme}>...</div>;
}
```

useMemo et useCallback — optimisation (à utiliser avec discernement)

```
// useMemo : mémorise un résultat de calcul coûteux
const resultatCalcule = useMemo(() => calculLourd(donnees), [donnees]);
```

```
// useCallback : mémorise une fonction (utile pour éviter de re-crée une fonction
// à chaque rendu, notamment si elle est passée à un composant enfant optimisé avec React.memo)
const handleClick = useCallback(() => {
  faireQuelqueChose(id);
}, [id]);
```

Bonne pratique : ne pas mettre `useMemo` / `useCallback` partout par réflexe — ça a un coût (mémoire, comparaison à chaque rendu) qui peut être pire que le problème qu'on essaie de résoudre. À utiliser quand un vrai problème de performance est mesuré, pas par anticipation systématique.

1.3 Props, composition, et flux de données

Une règle fondamentale, différente de la manipulation DOM jQuery où n'importe quel script peut modifier n'importe quel élément : **les données descendent (props), les événements remontent (callbacks)**.

```
function ListeArticles({ articles, onArticleClique }) {
  return (
    <ul>
      {articles.map(article => (
        <li key={article.id} onClick={() => onArticleClique(article.id)}>
          {article.nom}
        </li>
      ))}
    </ul>
  );
}

function PageCatalogue() {
  const [articles, setArticles] = useState([]);
  const [articleSelectionne, setArticleSelectionne] = useState(null);

  return (
    <ListeArticles
      articles={articles}
      onArticleClique={(id) => setArticleSelectionne(id)}
    />
  );
}
```

`ListeArticles` ne modifie jamais l'état du parent directement — elle appelle une fonction (`onArticleClique`) que le parent lui a fournie. C'est le parent qui décide quoi faire de cette information. Ce pattern ("callback prop") est omniprésent en React et sera tout aussi central en React Native.

1.4 Les listes et la prop `key`

```
{articles.map(article => (
  <li key={article.id}>{article.nom}</li>
))}
```

`key` doit être un identifiant **stable et unique** (l'ID en base, pas l'index du tableau sauf si la liste ne change jamais d'ordre). React utilise `key` pour savoir quel élément DOM correspond à quel élément de données entre deux rendus — sans ça, ou avec un index comme clé sur une liste qui peut être réordonnée/filtrée, tu obtiens des bugs d'affichage subtils (mauvais champ associé à la mauvaise ligne après un tri, par exemple), pas forcément une erreur visible immédiatement.

1.5 Bonnes pratiques actuelles (au-delà de la syntaxe)

Composants petits et spécialisés. Un composant qui fait 300 lignes et gère 5 responsabilités différentes est un signal à découper. Préfère plusieurs petits composants bien nommés à un gros

composant monolithique — exactement comme tu découperais un service Symfony trop large.

Lifting state up, mais pas trop haut. Si deux composants frères ont besoin de partager un état, on le "monte" dans leur parent commun le plus proche — pas systématiquement à la racine de l'app (qui mène à du `useContext` partout, dur à maintenir).

Éviter les états dérivés. Si une valeur peut être *calculée* à partir d'un autre état, ne la stocke pas dans un second `useState` — calcule-la directement au rendu (éventuellement avec `useMemo` si le calcul est coûteux). Stocker un état dérivé mène à des désynchronisations (le state "dérivé" qui ne se met pas à jour quand la source change).

```
// À éviter
const [articles, setArticles] = useState([]);
const [nombreArticles, setNombreArticles] = useState(0); // dérivé, risque de désync

// Préférer
const [articles, setArticles] = useState([]);
const nombreArticles = articles.length; // toujours synchronisé, calculé au rendu
```

Custom hooks pour réutiliser de la logique. Si plusieurs composants ont besoin de la même logique (appel API + état de chargement, par exemple), extrait-la dans un hook personnalisé plutôt que de dupliquer :

```
function useFetch(url) {
  const [donnees, setDonnees] = useState(null);
  const [chargement, setChargement] = useState(true);

  useEffect(() => {
    setChargement(true);
    fetch(url)
      .then(res => res.json())
      .then(data => {
        setDonnees(data);
        setChargement(false);
      });
  }, [url]);

  return { donnees, chargement };
}

// Utilisation, dans n'importe quel composant :
function MonComposant() {
  const { donnees, chargement } = useFetch('/api/articles');
  if (chargement) return <p>Chargement...</p>;
  return <pre>{JSON.stringify(donnees)}</pre>;
}
```

C'est l'équivalent conceptuel d'extraire une méthode réutilisable dans un service Symfony plutôt que de dupliquer la logique dans plusieurs contrôleurs.

(Fin de la Partie 1 — voir Partie 2 pour les spécificités React Native)

Partie 2 — React Native : ce qui change par rapport à React web

2.1 Le changement de paradigme : pas de DOM

C'est la différence la plus fondamentale, celle qui explique presque toutes les autres. React web génère du HTML, que le navigateur affiche via le DOM. React Native ne génère **aucun HTML** — il pilote directement des composants natifs iOS/Android (de vraies `UIView` iOS, de vraies `View` Android), via un pont JavaScript ↔ natif.

Conséquences concrètes : - Pas de `document.querySelector`, pas de `window`, pas de manipulation DOM même en JS pur - Pas de balises HTML : `<div>`, `<span>`, `<p>` n'existent pas — on utilise des composants fournis par React Native - Pas de CSS classique (pas de fichiers `.css`, pas de cascade, pas de sélecteurs)

2.2 Les composants de base

Web (HTML/React)	React Native	Rôle
<code><div></code>	<code><View></code>	Conteneur générique
<code></code> , <code><p></code> , <code><h1></code> ... <code><h6></code>	<code><Text></code>	Tout texte doit être enveloppé dans un <code><Text></code>
<code></code>	<code><Image></code>	<code>source={{ uri }}</code> (réseau) ou <code>source={require(...)}</code> (local)
<code><button></code>	<code><Pressable></code> ou <code><TouchableOpacity></code>	Pas de <code><button></code> natif
<code><input></code>	<code><TextInput></code>	<code>value</code> / <code>onChangeText</code> (pas <code>onChange</code> avec <code>event.target.value</code>)
Liste, conteneur scrollable	<code><ScrollView></code> ou <code><FlatList></code>	Voir 2.4

Règle absolue à retenir : si tu obtiens l'erreur *"Text strings must be rendered within a `<Text>` component"*, c'est que du texte brut traîne directement dans un `<View>` sans être enveloppé.

```
import { View, Text, Image, Pressable } from 'react-native';

function CarteArticle({ article, onPress }) {
  return (
    <Pressable onPress={onPress}>
      <View style={styles.carte}>
        <Image source={{ uri: article.imageUrl }} style={styles.image} />
        <Text style={styles.titre}>{article.nom}</Text>
        <Text style={styles.prix}>{article.prix}</Text>
      </View>
    </Pressable>
  );
}
```

2.3 Le style : StyleSheet, pas de CSS

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  carte: {
    flex: 1,
    flexDirection: 'column', // colonne PAR DÉFAUT (inverse du CSS web, où c'est 'row')
  }
});
```

```
padding: 16,
backgroundColor: '#ffffff',
borderRadius: 8,
},
titre: {
  fontSize: 18, // jamais d'unité – un nombre = density-independent pixels
  fontWeight: 'bold',
  color: '#222222',
},
});
```

Différences notables avec le CSS web :

Flexbox actif par défaut, partout. Pas besoin de `display: flex` — tous les conteneurs sont flex nativement. Mais `flexDirection` vaut `'column'` par défaut (le web utilise `'row'` par défaut) : piège classique pour qui vient du web, où l'on s'attend à un alignement horizontal par défaut.

Pas de cascade, pas d'héritage (sauf entre `<Text>` imbriqués, qui héritent des styles de texte entre eux). Chaque composant a son style propre, isolé — pas de sélecteurs CSS, pas de spécificité à gérer.

Combiner des styles se fait via un tableau, pas via plusieurs classes CSS :

```
<View style={[styles.carte, estSelectionne && styles.carteSelectionnee]} />
```

Pas d'unités : `fontSize: 18`, jamais `'18px'`. Tout est en density-independent pixels, géré automatiquement selon la densité d'écran de l'appareil.

Pas de pseudo-classes CSS (`:hover` n'a pas de sens sur mobile tactile ; `:active` se gère via les props `onPressIn` / `onPressOut` des composants tactiles).

2.4 Listes : `FlatList` vs `ScrollView`

`ScrollView` rend **tous** ses enfants d'un coup, même ceux hors écran — acceptable pour un contenu court (un formulaire, une page de détail), problématique en performance pour une longue liste.

`FlatList` est **virtualisée** : elle ne rend que les éléments visibles (+ une petite marge), recycle les composants au scroll. C'est l'équivalent d'une liste paginée/lazy côté web, mais natif et automatique.

```
import { FlatList } from 'react-native';

function ListeInterventions({ interventions, onSelectionner }) {
  return (
    <FlatList
      data={interventions}
      keyExtractor={({item}) => item.id.toString()} // équivalent de `key`, mais via une prop dédiée
      renderItem={({ item }) => (
        <CarteIntervention intervention={item} onPress={() => onSelectionner(item.id)} />
      )}
    />
  );
}
```

Piège fréquent : ne jamais imbriquer une `FlatList` à l'intérieur d'une `ScrollView` qui scrolle dans la même direction — ça casse le système de virtualisation et le scroll devient incohérent. Une seule liste scrollable principale par écran.

2.5 Navigation : React Navigation

Il n'y a pas d'URL en React Native, donc pas de React Router. La référence de l'écosystème est

React Navigation, qui modélise la navigation comme une pile d'écrans (proche du fonctionnement natif iOS/Android).

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Accueil">
        <Stack.Screen name="Accueil" component={EcranAccueil} />
        <Stack.Screen name="Detail" component={EcranDetail} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Naviguer et passer des paramètres :

```
// Dans EcranAccueil :
function EcranAccueil({ navigation }) {
  return (
    <Pressable onPress={() => navigation.navigate('Detail', { articleId: 42 })}>
      <Text>Voir le détail</Text>
    </Pressable>
  );
}

// Dans EcranDetail :
function EcranDetail({ route }) {
  const { articleId } = route.params;
  // ...
}
```

useFocusEffect — hook spécifique à la navigation, à connaître : s'exécute chaque fois que l'écran redevient actif (utile pour rafraîchir une liste quand on revient d'un écran de détail, par exemple) :

```
import { useFocusEffect } from '@react-navigation/native';
import { useCallback } from 'react';

function EcranListe() {
  useFocusEffect(
    useCallback(() => {
      rechargerLesDonnees();
    }, [])
  );
  // ...
}
```

Différence avec **useEffect** classique : **useEffect** avec `[]` ne s'exécute qu'au tout premier montage du composant — si l'écran reste monté en arrière-plan dans la pile de navigation (cas fréquent), il ne se redéclenche pas en revenant dessus. **useFocusEffect** si.

2.6 Hooks : ce qui ne change pas (et c'est une bonne nouvelle)

useState, **useEffect**, **useContext**, **useMemo**, **useCallback**, et tes futurs hooks personnalisés fonctionnent **à l'identique** de React web. Toute la logique d'état et la logique métier que tu écris se porte quasiment sans changement entre web et natif — c'est uniquement la couche de *rendu visuel* (composants + style) qui diffère.

C'est une excellente nouvelle pour ta migration : ta logique de synchronisation avec l'API Symfony, tes hooks de gestion d'état, peuvent être repris presque tels quels.

2.7 Stockage local

`localStorage` n'existe pas en React Native (pas de DOM, pas de navigateur). L'équivalent standard est `AsyncStorage` :

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Écriture (toujours asynchrone, retourne une Promise)
await AsyncStorage.setItem('derniere_sync', new Date().toISOString());

// Lecture
const valeur = await AsyncStorage.getItem('derniere_sync');
```

Différence clé avec `localStorage` : tout est asynchrone (Promises), jamais synchrone.

2.8 Appels réseau

`fetch` fonctionne nativement et de façon identique au web — c'est une autre bonne nouvelle pour ta migration, ta logique d'appel API (synchronisation des interventions) se porte presque sans changement :

```
async function recupererInterventions(salonId, jour) {
  const reponse = await fetch(`https://ton-api.fr/interventions?salon=${salonId}&jour=${jour}`);
  if (!reponse.ok) {
    throw new Error(`Erreur API : ${reponse.status}`);
  }
  return await reponse.json();
}
```

2.9 Accès aux fonctionnalités natives (caméra, signature, etc.)

Pour tout ce qui touche au matériel ou à des composants UI très spécifiques (signature tactile, caméra, géolocalisation, notifications push), on passe par des **librairies tierces** packageant le pont natif iOS/Android :

```
import SignatureScreen from 'react-native-signature-canvas';

function EcranSignature({ onSignatureCapturee }) {
  const handleOK = (signatureBase64) => {
    onSignatureCapturee(signatureBase64); // chaîne "data:image/png;base64,..."
  };

  return <SignatureScreen onOK={handleOK} />;
}
```

Le pattern général : la lib expose un composant React (donc tu l'utilises en JSX normalement), mais en coulisse elle communique avec du vrai code natif (Swift/Objective-C côté iOS, Kotlin/Java côté Android) via le pont React Native. Tu n'as presque jamais besoin d'écrire ce code natif toi-même pour des besoins courants — l'écosystème de libs est très riche.

2.10 Platform — gérer les différences iOS / Android

Certains comportements/styles diffèrent légèrement entre les deux plateformes (ombres, certains comportements de clavier, etc.) :

```
import { Platform, StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  carte: {
    ...Platform.select({
      ios: { shadowColor: '#000', shadowOpacity: 0.1, shadowRadius: 4 },
      android: { elevation: 3 },
    }),
  },
});
```

```
},  
});
```

```
if (Platform.OS === 'ios') {  
  // logique spécifique iOS  
}
```

À utiliser ponctuellement, pas systématiquement — la plupart du code reste identique entre les deux plateformes.

2.11 Workflow de développement : Expo

Expo est aujourd'hui le moyen le plus simple de démarrer et d'itérer sur un projet React Native, sans configuration native lourde au départ.

Expo Go (app à installer sur ton téléphone) permet de voir ton app en cours de développement en scannant un QR code généré par :

```
npx expo start
```

Le rechargement est quasi instantané à chaque sauvegarde (Fast Refresh), comme le Hot Reload auquel tu es habitué côté web.

Limite à connaître : Expo Go ne supporte pas *tous* les modules natifs tiers (certains nécessitent un "build natif personnalisé", via `expo prebuild` ou `eas build`). Pour un projet avec des besoins spécifiques (signature tactile, notifications avancées), il est fréquent de devoir basculer vers un build de développement personnalisé à un moment du projet — mais Expo gère ça aussi, juste avec une étape supplémentaire.

2.12 Différences de mentalité à intégrer

Pas de "vue" qui s'adapte au contenu par défaut comme le flux HTML normal. En CSS web, un `<div>` vide ne prend pas de place. En React Native, sans `flex` ou dimensions explicites bien pensées, on se retrouve facilement avec des écrans vides ou des éléments qui ne s'affichent pas — il faut penser la mise en page comme un arbre de conteneurs flexibles dès le départ, pas comme un flux de blocs qui s'empilent naturellement.

Le clavier mobile mange une partie de l'écran. Quand un `<TextInput>` reçoit le focus, le clavier virtuel apparaît et réduit l'espace visible — il faut souvent gérer ça explicitement (composant `KeyboardAvoidingView`), un problème qui n'existe simplement pas sur desktop web.

Tester sur les deux plateformes régulièrement. Un comportement parfaitement correct sur iOS peut avoir une nuance différente sur Android (ombre, comportement de clavier, gestes). Ne pas attendre la fin du projet pour tester sur l'autre plateforme.

Synthèse : ce qui se porte facilement vs ce qui se réécrit

Élément	Portabilité depuis ton code Preact/web
Logique d'état (<code>useState</code> , <code>useEffect</code> , hooks custom)	✓ Quasi identique
Appels API (<code>fetch</code> , logique de synchronisation)	✓ Quasi identique
Structure des données, logique métier pure	✓ Identique
Composants visuels (JSX avec <code><div></code> , <code></code> ...)	✗ À réécrire avec les composants RN
Styles CSS	✗ À réécrire en <code>StyleSheet</code>
Navigation (routing)	✗ À réécrire avec React Navigation
Stockage local	⚠ <code>localStorage</code> → <code>AsyncStorage</code> (API différente, asynchrone)
Formulaire SurveyJS (signature)	✗ À remplacer par un composant natif dédié

Cette répartition explique pourquoi une migration Cordova/Preact → React Native, bien menée, conserve une bonne partie de la valeur métier déjà écrite (logique de synchronisation, gestion d'état) tout en demandant une réécriture complète de la couche de présentation.